

基于RISC-V代理内核的操作 系统课程实验与课程设计

第三章. 实验1: 系统调用、异常和外部中断

目录

- 实验1的基础知识
 - RISC-V程序的编译和链接
 - 指定符号的逻辑地址
 - 代理内核的构造过程
 - 代理内核的启动过程
 - ELF文件（app）的加载过程
 - spike的HTIF接口
- 实验内容
 - lab1_1 系统调用
 - lab1_2 异常处理
 - lab1_3（外部）中断

3.1 实验1的基础知识

RISC-V程序的编译过程

RISC-V程序编译主要过程：

- 预处理：展开头文件、宏替换、条件编译、删除注释等等；
- 编译：语法检查；
- 汇编：将汇编文件翻译成机器码（二进制文本），生成目标文件；
- 链接：将所有的目标文件打包链接成一个可执行文件。

指定符号的逻辑地址

使用链接脚本，例如：

```
OUTPUT_ARCH( "riscv" )
```

```
ENTRY(main)
```

```
SECTIONS
```

```
{
```

```
    . = 0x81000000;
```

```
    . = ALIGN(0x1000);
```

```
    .text : { *(.text) }
```

```
    . = ALIGN(16);
```

```
    .data : { *(.data) }
```

```
    . = ALIGN(16);
```

```
    .bss : { *(.bss) }
```

```
}
```

代理内核的构造（build）过程

采用makefile，简化构造过程，此处摘取makefile的一部分。

```
KERNEL_TARGET = $(OBJ_DIR)/riscv-pke
```

```
USER_TARGET := $(OBJ_DIR)/app_helloworld
```

```
.....
```

```
$(KERNEL_TARGET): $(OBJ_DIR) $(UTIL_LIB) $(SPIKE_INF_LIB) $(KERNEL_OBJS) $(KERNEL_LDS)
```

```
    @echo "linking" $@ ...
```

```
    @$(COMPILE) $(KERNEL_OBJS) $(UTIL_LIB) $(SPIKE_INF_LIB) -o $@ -T $(KERNEL_LDS)
```

```
    @echo "PKE core has been built into" \"$@\"
```

```
.....
```

```
.DEFAULT_GOAL := $(all)
```

```
all: $(KERNEL_TARGET) $(USER_TARGET)
```

```
.PHONY:all
```

代理内核的构造（build）过程

构造过程可概括为：

1. 构造util目录下的静态库文件\$(OBJ_DIR)/util.a;
2. 构造应用程序，得到\$(OBJ_DIR)/app_helloworld;
3. 构造\$(OBJ_DIR)/spike_interface.a，即spike所提供的工具库文件；
4. 最后构造代理内核\$(OBJ_DIR)/riscv-pke。

代理内核的启动过程

采用spike模拟器（执行命令）：

```
spike ./obj/riscv-pke ./obj/app_helloworld
```

启动具体过程：

1. 在M态执行一系列初始化动作（如HTIF）；
2. 将中断异常处理“代理”给S态，进入S态；
3. 加载程序到内存，将程序封装成一个最简单的“进程”，然后调度执行，同时进入U态。

ELF文件（**app**）的加载过程

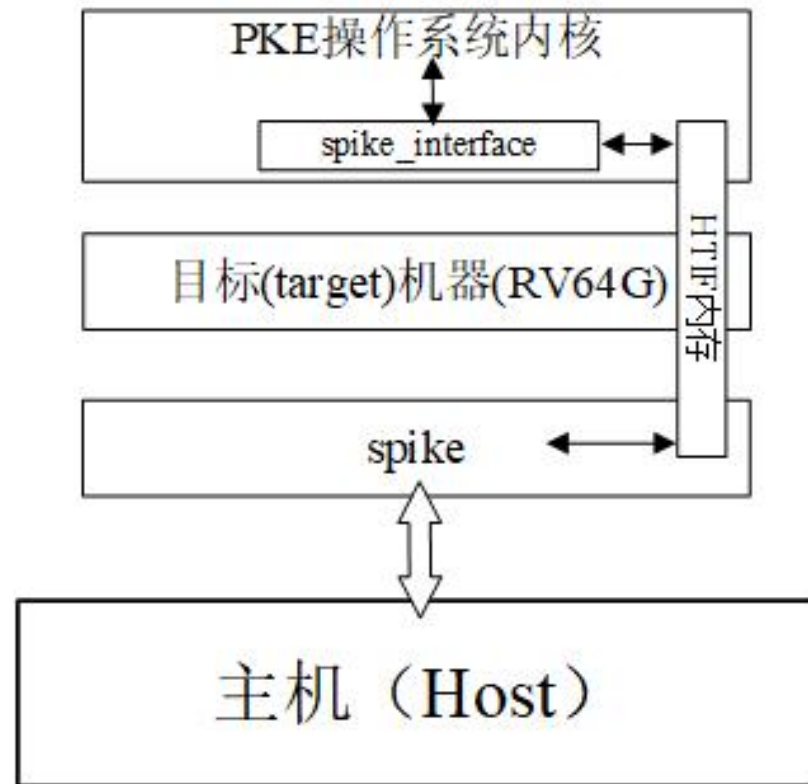
大致过程为：

1. 首先，解析命令行参数，获得需要加载的ELF文件文件名；
2. 接下来初始化ELF加载数据结构，并打开即将被加载的ELF文件；
3. 加载ELF文件；
4. 通过ELF文件提供的入口地址设置进程的“入口地址”，保证“返回”用户态的时候，所加载的ELF文件被执行；
5. 关闭ELF文件并返回。

spike的HTIF接口

HTIF（Host Target Interface）是一种通信协议，负责主机与代理内核的通信。

主要功能：读/写内存、加载/启动/停止程序等等。



HTIF原理示意图

中断处理例程

- 当发生一个目标模式为机器模式的中断，RISC-V处理器硬件将执行以下动作：
 - 1) 保存pc到mepc寄存器；
 - 2) 将特权级保存到mstatus寄存器的MPP字段；
 - 3) 将mstatus寄存器中的MIE字段保存到（它自己的）MPIE字段；
 - 4) 设置mcause，其值与中断向量表中的Interrupt和Exception code对应；
 - 5) 将pc设置为中断例程的入口，如果为直接模式则设置为mtvec的值；
 - 6) 将mstatus寄存器的MIE字段清零，转入机器模式。

中断处理例程

注：在PKE实验中，系统的中断实际上是代理给监管模式处理的，在发生中断时处理器硬件的流程与以上的机器模式类似，只是mepc、mstatus、mcause以及mtvec换成了sepc、sstatus、scause以及stvec。

典型的中断处理过程

```
.align 6
.global handler_interrupt
handler_interrupt:
addi sp, sp, -32*REGBYTES
STORE x1, 1* REGBYTES(sp)
...
STORE x31, 31* REGBYTES(sp)
//call C code handler
call software_handler
//finished interrupt handling, ready to return
LOAD x1, 1* REGBYTES(sp)
...
LOAD x31, 31* REGBYTES(sp)
addi sp, sp, 32*REGBYTES
mret
```

注：处理器在执行mret指令时，将执行以下动作：

- 1) 将mstatus寄存器的MPIE字段恢复到该寄存器的MIE字段；
- 2) 处理器转换到mstatus寄存器中MPP字段所对应的特权模式；
- 3) 将mepc中的内容恢复到pc中。

目录

- 实验1的基础知识
 - RISC-V程序的编译和链接
 - 指定符号的逻辑地址
 - 代理内核的构造过程
 - 代理内核的启动过程
 - ELF文件（app）的加载过程
 - spike的HTIF接口
- 实验内容
 - lab1_1 系统调用
 - lab1_2 异常处理
 - lab1_3（外部）中断

lab1_1 系统调用

- user/app_helloworld.c

```
1 /*
2  * Below is the given application for lab1_1.
3  *
4  * You can build this app (as well as our PKE OS kernel) by command:
5  * $ make
6  *
7  * Or run this app (with the support from PKE OS kernel) by command:
8  * $ make run
9  */
10
11 #include "user_lib.h"
12
13 int main(void) {
14     printu("Hello world!\n");
15
16     exit(0);
17 }
```

给定应用

如输出提示所表示的那样，需要找到并完成对do_syscall的调用，并获得以下预期结果：

```
$ spike ./obj/riscv-pke ./obj/app_helloworld
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_helloworld
Application program entry point (virtual address): 0x0000000081000000
Switching to user mode...
Hello world!
User exit with code:0.
System is shutting down with exit code 0.
```

预期输出

lab1_1 系统调用

实验内容：

给定应用没有实现预期效果，即在屏幕上打印“Hello world!\n”。

实验代码中syscall没有被调用，找到并完成对do_syscall的调用，获得预期的输出结果。需要通过调用handle_illegal_instruction函数完成异常指令处理，阻止app_illegal_instruction的执行。

lab1_2 异常处理

- user/app_illegal_instruction.c

```
1 /*
2  * Below is the given application for lab1_2.
3  * This app attempts to issue M-mode instruction in U-mode, and consequently raises an exception.
4  */
5
6 #include "user_lib.h"
7 #include "util/types.h"
8
9 int main(void) {
10     printf("Going to hack the system by running privilege instructions.\n");
11     // we are now in U(user)-mode, but the "csrw" instruction requires M-mode privilege.
12     // Attempting to execute such instruction will raise illegal instruction exception.
13     asm volatile("csrw sscratch, 0");
14     exit(0);
15 }
```

给定应用

如输出所提示的那样，通过调用handle_illegal_instruction函数完成异常指令处理

```
$ spike ./obj/riscv-pke ./obj/app_illegal_instruction
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_illegal_instruction
Application program entry point (virtual address): 0x0000000081000000
Switching to user mode...
Going to hack the system by running privilege instructions.
Illegal instruction!
System is shutting down with exit code -1.
```

预期输出

lab1_2 异常处理

实验内容：

（在用户U模式下执行的）应用企图执行RISC-V的特权指令 `csrw sscratch, 0`。该指令会修改S模式的栈指针，如果允许该指令的执行，执行的结果可能会导致系统崩溃。

通过调用 `handle_illegal_instruction` 函数完成异常指令处理，阻止 `app_illegal_instruction` 的执行。

lab1_3 （外部） 中断

实验完成后的运行结果：

- user/app_long_loop.c

```
1 /*
2  * Below is the given application for lab1_3.
3  * This app performs a long loop, during which, timers are
4  * generated and pop messages to our screen.
5  */
6
7 #include "user_lib.h"
8 #include "util/types.h"
9
10 int main(void) {
11     printu("Hello world!\n");
12     int i;
13     for (i = 0; i < 100000000; ++i) {
14         if (i % 5000000 == 0) printu("wait %d\n", i);
15     }
16
17     exit(0);
18
19     return 0;
20 }
```

给定应用

```
$ spike ./obj/riscv-pke ./obj/app_long_loop
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: obj/app_long_loop
Application program entry point (virtual address): 0x000000008100007e
Switching to user mode...
Hello world!
wait 0
wait 5000000
wait 10000000
Ticks 0
wait 15000000
wait 20000000
Ticks 1
wait 25000000
wait 30000000
wait 35000000
Ticks 2
wait 40000000
wait 45000000
Ticks 3
wait 50000000
wait 55000000
wait 60000000
```

预期输出

lab1_3 （外部） 中断

实验内容：

给定的程序在运行过程中受到了系统的外部时钟中断（timer irq）的“干扰”。而我们在这个实验中给出的PKE操作系统内核，在时钟中断部分并未完全做好，导致（模拟）RISC-V机器碰到第一个时钟中断后就会出现崩溃。

完成PKE操作系统内核未完成的时钟中断处理过程，使得它能够完整地处理时钟中断。